SYSTEM DESIGN

Design YouTube

A global video streaming platform handling 2B+ users, 500 hours/minute uploads, 1B hours/day watched

~ 45 minute read

© Study Guide — For personal use only

What's Inside

01 · Problem Statement & Clarifying Questions
02 · Functional & Non-Functional Requirements
03 · Capacity Estimation & Metrics
04 · High-Level Architecture
05 · Video Upload & Ingestion
06 · Transcoding Deep Dive
07 · Adaptive Bitrate Streaming
08 · CDN Architecture
09 · Database Design
10 · Video Search
11 · Recommendation Engine
12 · Comments & Engagement
13 · Caching Strategy
14 · Failure Modes & Resilience
15 · Trade-Offs & Design Decisions
16 · Interview Playbook

01 Problem Statement

Ask 7 clarifying questions

You are asked to design YouTube, a global video streaming platform where users can upload, watch, and share videos. The interviewer expects you to start with clarifying questions to scope the problem.

Key Clarifying Questions

Question	Answer / Scope
1. Scale: how many users?	2 billion total users; ~500 million monthly active users
2. Video upload volume?	500 hours of video uploaded per minute globally
3. Video storage duration?	Indefinite (at least 10 years); some pruning of very old unwatched content
4. Geographic reach?	Global; must support playback in all regions with local CDN
5. Real-time features?	Live streaming out of scope for MVP; focus on stored video on-demand
6. Video resolution?	Support multiple: from 144p (mobile) to 4K; user device adaptivity
7. Core product focus?	Watch experience > upload experience. Search, recommendations, comments secondary.

Design Constraints (from clarifications)

- **Global scale:** 500M MAU spread across 6 continents → need geographically distributed CDN
- **Upload velocity:** 500 h/min = 30,000 h/day = 1.25M hours/day → massive ingestion pipeline
- **Watch volume:** ~1 billion video hours watched per day globally (estimated)
- **Bias towards playback:** Read-heavy system (watch:upload ratio ≈ 2000:1)
- **Quality expectations:** Viewers expect <2-second startup time, zero buffering, instant pause/resume

© Study Guide — For personal use only

02 Functional & Non-Functional Requirements

Functional Requirements

- Upload video: chunk-based resumable upload with progress tracking
- Watch video: stream with ABR (adaptive bitrate), low latency, seek support
- List videos: search by title/tags, browse by category, trending videos
- Recommend videos: personalized recommendations based on watch history & collaborative filtering
- Comments: post, reply, like comments; nested threads with pagination
- Subscriptions: subscribe to channels, get notifications for new uploads

Non-Functional Requirements

Metric	Target
Video startup latency	<2 seconds (p99)
Rebuffer rate	<0.5% of streams
Availability	99.99% (4 nines)
Search latency	<100ms (p99)
Recommendation latency	<500ms (p99)
Storage durability	99.999999999% (11 nines, triple replication)
Video encoding latency	<4 hours for 1 TB (≈ 2 Mbps processing)

© Study Guide — For personal use only

03 Capacity Estimation

Calculate QPS, storage, bandwidth

Daily Upload Volume

```
500 hours/minute × 60 minutes = 30,000 hours/day

Assuming avg video is 10 minutes:
30,000 hours = 30,000 × 60 min = 1,800,000 videos/day

Upload QPS (assuming spread over 24h = 86,400 seconds):
1,800,000 / 86,400 ≈ 21 video_uploads/second
But peak-hour ≈ 3× average = ~60-70 uploads/sec during peak
```

Daily Storage Requirement

```
Raw video storage:
30,000 hours/day × 1 GB/hour = 30 TB raw video/day

Transcoded (10 formats: 4K+1080p+720p+480p+360p+240p+144p,
each in H.264/VP9):
30 TB × ~10 transcoded formats ≈ 300 TB/day

Annual storage needed:
300 TB/day × 365 = 109.5 PB/year
~5-year retention = 547 PB (assuming linear growth)

With 3× replication for durability:
547 PB × 3 = ~1.64 EB (Exabyte)
```

Streaming (Watch) Volume & Bandwidth

```
Estimated daily watch hours: ~1 billion hours/day

Concurrent streams at peak:
1B hours/day ÷ 86,400 sec = 11.6 million concurrent viewers

Bandwidth requirement (at 2 Mbps average adaptive bitrate):
11.6M streams × 2 Mbps = 23.2 Tbps (Terabits/sec) peak

CDN needs to handle ≈2.3 Pbps global bandwidth
Accounting for PoP peering, this is ≈ 500-1000 Tbps to origin.
```


© Study Guide — For personal use only

```
Cache efficiency: ~95% hit rate on CDN edges → only 5% origin pulls
```

Database Metrics

- Videos table: ~7 billion total videos (30K/day × 10 years); ~500 bytes/row = 3.5 TB
- Users table: ~2 billion users; ~1 KB/row = 2 TB
- Comments: ~50 billion comments (assume 7:1 comment:video ratio); Cassandra sharded
- Likes: ~500 billion likes across videos & comments; Redis counters + async flush

© Study Guide — For personal use only

04 High-Level Architecture

Multi-tier service design

The system is divided into several layers: client, edge/CDN, API, core services, and data tier. Each layer is independently scalable and fault-tolerant.

© Study Guide — For personal use only

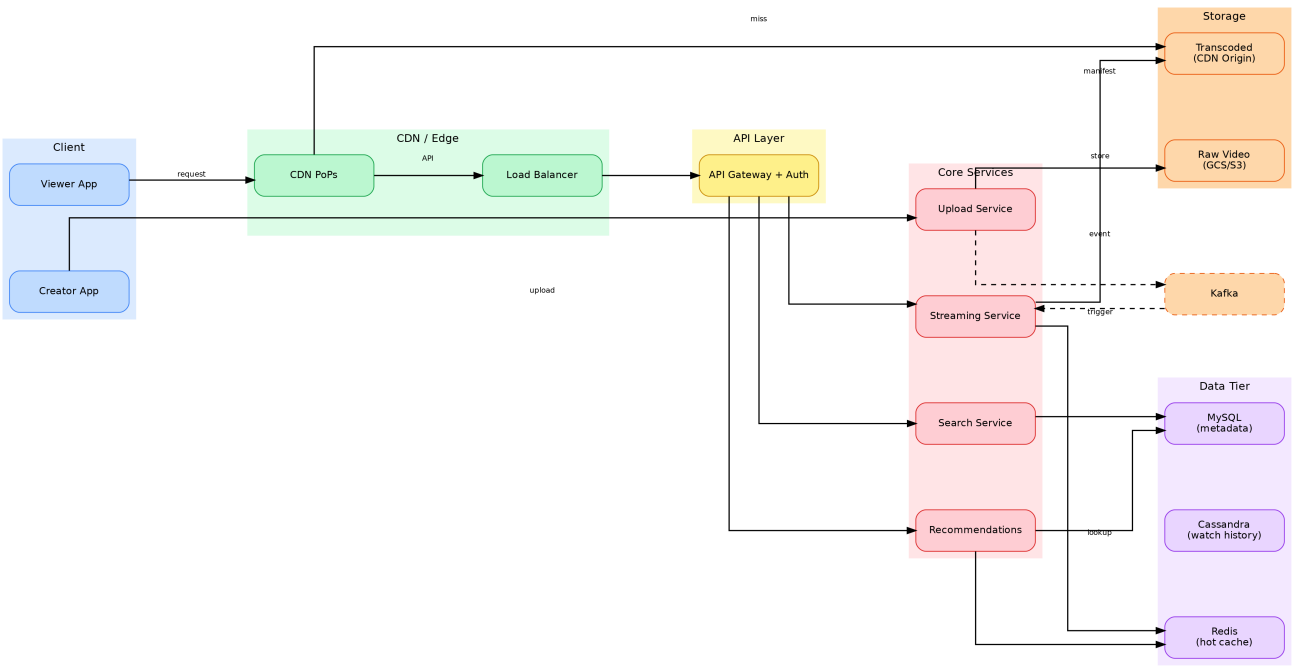

Figure 4.1: High-Level System Architecture — Viewer and creator traffic flows through distinct paths for efficient scaling.

Component Overview

- **Client:** Web browser, mobile app, smart TV; initiates playback requests or uploads
- **Load Balancer:** Distributes inbound requests across API gateways
- **API Gateway:** Authentication, rate limiting, request routing
- **Upload Service:** Handles chunked/resumable uploads (TUS protocol)
- **Streaming Service:** Serves HLS/DASH manifests and segments
- **Search Service:** Elasticsearch for title/tag search with ranking
- **Recommendation Service:** Two-tower model (candidate generation + ranking)
- **Transcoding Cluster:** Parallel workers encoding videos into multiple formats
- **Message Queue (Kafka):** Event-driven pipeline: upload_done → transcode_trigger
- **S3 / GCS:** Raw video storage (ingest) and transcoded segments (streaming)
- **CDN PoPs:** Geographically distributed Points-of-Presence caching video segments
- **MySQL:** Transactional metadata (videos, users, metadata)
- **Cassandra:** Time-series comments, scales horizontally
- **Redis:** Caching, session store, real-time counters (likes, views)

> **N INSIGHT**
>
> The architecture separates **write path** (upload → transcode) from **read path** (stream from CDN). This allows independent scaling of each.

 © Study Guide — For personal use only

05 Video Upload & Ingestion

Resumable uploads, deduplication

© Study Guide — For personal use only

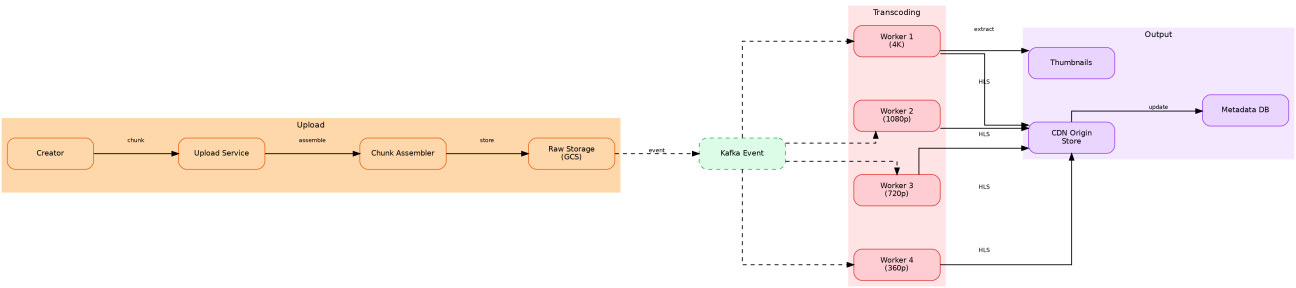

Figure 5.1: Video Upload & Transcoding Pipeline — Raw video is ingested, assembled, and dispatched to parallel transcoding workers.

TUS Protocol (Resumable Upload)

The Tus protocol (https://tus.io/) standardizes resumable uploads. YouTube-scale requires:

- **Chunking:** Split video into 5-10 MB chunks (standard chunk size)
- **Resumability:** Track which chunks uploaded; client can resume from last chunk on network failure
- **Server acknowledgment:** Upload Service returns offset and next chunk index
- **Atomic commits:** Only mark upload complete when all chunks received and checksummed

Upload Flow (Step-by-Step)

1. **Creator initiates upload:** POST /upload → receives upload_id, chunk size hints
2. **Client chunks and sends:** PATCH /upload/{id}/chunk/{n} with chunk + checksum
3. **Server acknowledges:** Returns offset progress; client buffers retries on transient error
4. **Upload completes:** Final chunk triggers verification; checksum mismatch → re-upload chunk
5. **Kafka event published:** upload_complete {video_id, creator_id, original_size} → event log
6. **Transcoding orchestrator:** Consumes event, dispatches tasks to transcoding workers
7. **Raw video stored:** S3 at s3://raw-videos/{creator_id}/{video_id}.mp4

Deduplication & Content-Addressed Storage

To avoid duplicate storage of identical videos (copies, reuploads):

- **Perceptual hashing:** Compute PHASH of uploaded video during transcoding
- **PHASH lookup:** Check if PHASH exists in Cassandra; if match found → link video_id to existing content
- **Space savings:** Avoid 50+ TB/day duplicate storage (estimated 10-15% duplicate rate)
- **Copyright enforcement:** Identical PHASHes flag potential copyright violations for review team

Handling Upload Failures

- **Transient network failure:** Client retries chunk with exponential backoff
- **Timeout on chunk:** Upload Service holds state; client resumes after 5min
- **Corrupted chunk (checksum mismatch):** Client re-sends; server re-computes checksum

 © Study Guide — For personal use only

- **Upload abandoned:** Garbage collector cleans up incomplete uploads after 7 days

> ■ **WARNING**
>
> YouTube accepts uploads from anyone. Abuse mitigation: initial uploads placed in **moderation queue** for bot detection (content-hash lookup, NSFW classifier). Cleared uploads go to transcoding.

 © Study Guide — For personal use only

06 Transcoding

Codec selection, resolution ladder, cost

Codec Comparison

Codec	Bitrate (1080p)	Compression	Adoption	Notes
H.264 (AVC)	3-5 Mbps	Good (2003)	Universal (99.9%)	Highest compatibility; patent pool licenses
VP9	2-3 Mbps	Excellent	~80% devices	Open-source; better compression than H.264
AV1	1-1.5 Mbps	Excellent+	Emerging (~40%)	Next-gen; high CPU cost; 5-10× encoding time
HEVC (H.265)	2-3 Mbps	Excellent	~75% devices	Patent-heavy licensing; superior quality at bitrate

YouTube's Strategy

- Primary: H.264 (universal fallback for all browsers)
- Secondary: VP9 (Chrome, Firefox; saves ~30% bandwidth)
- Tertiary: HEVC (Apple devices; iOS/macOS, saves ~40% vs H.264)
- Future: AV1 for premium content (4K+); encoding cost too high for bulk

Resolution Ladder

```
Standard ladder (10 resolutions × 2-3 codecs):

4K (2160p): H.264 (8-12 Mbps), VP9 (5-7 Mbps), HEVC (4-6 Mbps)
1440p: H.264 (5-8 Mbps), VP9 (3-5 Mbps), HEVC (2.5-4 Mbps)
1080p: H.264 (3-5 Mbps), VP9 (2-3 Mbps), HEVC (1.5-2.5 Mbps)
720p: H.264 (2-3 Mbps), VP9 (1-2 Mbps), HEVC (1-1.5 Mbps)
480p: H.264 (1-1.5 Mbps), VP9 (0.8-1.2 Mbps)
360p / 240p / 144p: H.264 only (mobile-optimized, lowest bitrate)
```

Transcoding Cost & Performance

- **CPU-intensive:** 1 hour of 1080p H.264 encoding ≈ 45 min CPU @ 8-core machine

© Study Guide — For personal use only

- **Parallel strategy:** 8 workers per node; each encodes different resolution simultaneously
- **Cost optimization:** ~$0.015 per hour of video transcoded (compute + storage + bandwidth)
- **SLA:** >90% of uploads transcoded within 4 hours; <10% within 24 hours

© Study Guide — For personal use only

07 Adaptive Bitrate Streaming

ABR algorithms, manifest format

© Study Guide — For personal use only

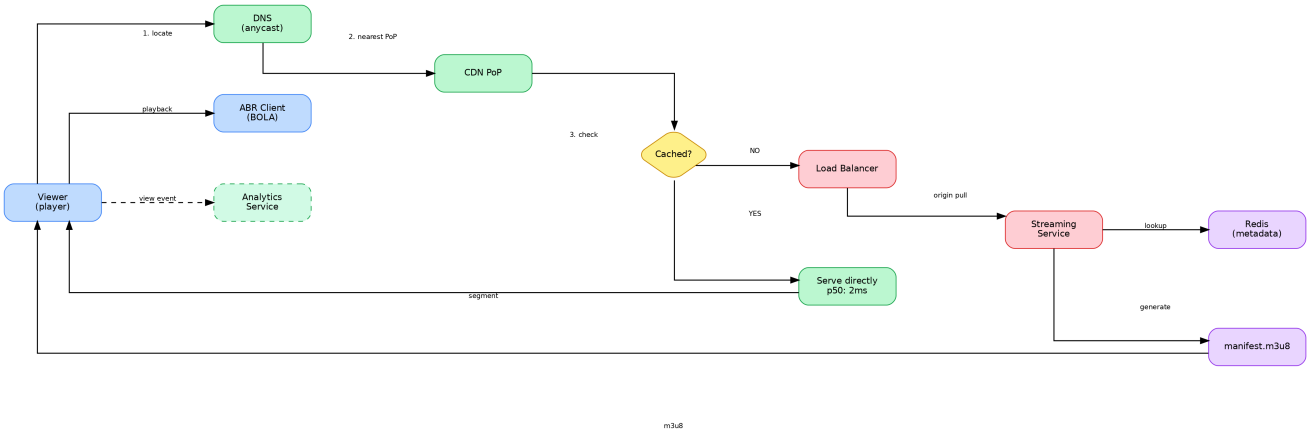

Figure 7.1: Video Streaming Request Flow — Viewer requests manifest, player dynamically selects segments based on bandwidth.

HLS (HTTP Live Streaming) Format

```
#EXTM3U
#EXT-X-STREAM-INF:BANDWIDTH=2500000,RESOLUTION=1280x720
720p/segment_list.m3u8
#EXT-X-STREAM-INF:BANDWIDTH=1500000,RESOLUTION=854x480
480p/segment_list.m3u8
#EXT-X-STREAM-INF:BANDWIDTH=500000,RESOLUTION=426x240
240p/segment_list.m3u8
```

Example master.m3u8: lists available quality levels

Adaptive Bitrate (ABR) Algorithms

Algorithm	Strategy	Latency	Fairness
BOLA (Buffer-based)	Maximize quality while maintaining buffer	Low	High
Pensieve (ML-based)	RL model predicts optimal bitrate	Very low	Very high
Rate-based	Adjust bitrate based on network probe	Medium	Medium
Buffer-filling	Ramp up quality until buffer > threshold	High	Low

BOLA (Buffer-Optimal Latency-Aware)

```
BOLA uses a utility function to maximize quality while avoiding stalls:

Q(r) = log(bitrate[r]) − stall_penalty × P(stall | buffer, rate)
```

 © Study Guide — For personal use only

```
Decision: select highest Q(r) such that buffering won't occur.

Advantages:
• Provably optimal in offline setting
• Stable — rarely switches bitrate (reduces audio/visual artifacts)
• Fairness: balances competing streams on shared bandwidth
```

Manifest Request Flow

1. **Player requests manifest:** GET /manifest?video_id=abc&device;=mobile
2. **Streaming Service resolves:** Redis cache hit (99%); returns cached .m3u8
3. **Cache miss (<1%):** Hits MySQL → retrieves list of quality tiers + segment URLs
4. **Manifest sent to player:** Includes master.m3u8 with bandwidth hints
5. **Player selects quality:** BOLA algorithm picks quality ≤ current bandwidth estimate
6. **Segment fetch:** GET /segments/720p_seg001.ts → CDN PoP (95% hit rate ✓)
7. **Adaptive switch:** Every 1-3 segments, player re-evaluates buffer + bandwidth

© Study Guide — For personal use only

08 CDN Architecture

Multi-tier caching, geo-distribution

CDN Tier Strategy

- **Tier 1 (Edge PoPs):** Closest to users; cache popular content (hot videos)
- **Tier 2 (Regional):** Larger caches; serves as fallback for Tier 1 misses
- **Tier 3 (Origin):** Primary S3 / object storage; holds all transcoded segments

Cache Optimization

- **Hit rate targets:** 95-99% at edge; 99.5%+ at regional
- **Popular content:** Pre-warm Tier 1 for trending videos (auto-detected by analytics)
- **TTL strategy:** Long-lived content (>30 days) → TTL 365d; new content → TTL 7d
- **Purge on update:** If creator re-uploads thumbnail/metadata, invalidate cache entries

Geo-Distribution

```
Estimated PoP locations (simplified):

North America: ~50 PoPs (NYC, LA, Chicago, Denver, etc.)
Europe: ~40 PoPs (London, Paris, Frankfurt, Moscow, etc.)
Asia-Pacific: ~60 PoPs (Tokyo, Singapore, Sydney, Delhi, etc.)
South America: ~15 PoPs (São Paulo, Buenos Aires, etc.)
Africa: ~10 PoPs (Johannesburg, Cairo, etc.)

Total: ~175 PoPs globally
```

Traffic Flow & Load Balancing

- **DNS Anycast:** User resolves video.youtube.com → closest PoP via GeoDNS
- **HTTP Redirect:** If PoP full / low capacity, redirect to next-nearest PoP
- **Load balancing:** Round-robin across PoP servers; monitor CPU / bandwidth per server

© Study Guide — For personal use only

09 Database Design

Schema, sharding, consistency

Primary Databases

- **MySQL (Videos, Users):** Strongly consistent; transactional; ACID properties for financial/critical data
- **Cassandra (Comments, Watch History):** Eventually consistent; high write throughput; distributed sharding
- **Redis (Cache, Counters):** In-memory; sub-millisecond latency; volatile but acceptable for ephemeral data
- **Elasticsearch (Search Index):** Full-text indexing; ranked search; refreshed every 30 seconds

MySQL Schema (Simplified)

```
CREATE TABLE videos (
video_id BIGINT PRIMARY KEY,
creator_id BIGINT NOT NULL,
title VARCHAR(500) NOT NULL,
description TEXT,
duration INT, -- seconds
created_at TIMESTAMP,
updated_at TIMESTAMP,
status ENUM('processing', 'ready', 'blocked'),
manifest_url VARCHAR(255), -- s3 origin
thumbnail_url VARCHAR(255),
view_count BIGINT DEFAULT 0,
like_count BIGINT DEFAULT 0,
INDEX idx_creator (creator_id),
INDEX idx_status (status)
);
```

Cassandra Schema (Watch History)

```
CREATE TABLE watch_history_by_user (
user_id BIGINT,
watched_at BIGINT, -- timestamp in millis (sorted DESC)
video_id BIGINT,
watched_duration INT,
quality VARCHAR(10),
```


© Study Guide — For personal use only

```
PRIMARY KEY (user_id, watched_at)
) WITH CLUSTERING ORDER BY (watched_at DESC);
-- Distributed: sharded by user_id
```

Sharding Strategy

- **MySQL (Videos):** Shard by video_id hash; ~10-20 shards (growth headroom)
- **Cassandra (Comments):** Shard by video_id; each node replicates 3 copies
- **Consistency requirement:** MySQL = strong; Cassandra = eventual (OK for comments)

© Study Guide — For personal use only

10 Video Search

Elasticsearch, ranking, autocomplete

Full-Text Search with Elasticsearch

- **Index:** Video metadata (title, description, tags, category)
- **Indexing latency:** ~30 seconds (near-real-time); updated via Kafka topic
- **Sharding:** 10 primary shards + 2 replicas per shard
- **Typical query:** 'python programming' → ranked by relevance, upload date, popularity

Search Ranking Formula

```
score(video) = TF-IDF(query, title) × 0.4
+ TF-IDF(query, description) × 0.2
+ BM25(query, tags) × 0.2
+ popularity_score(view_count, likes) × 0.15
+ recency_boost(1 - age_in_days / 365)

Results sorted by score DESC, with freshness boost for recent videos
```

Autocomplete / Suggestions

- **Data source:** Top 1M search queries (aggregated from analytics)
- **Prefix tree (trie):** Stores common query prefixes with hit count
- **Latency:** <100ms (p99); served from Redis + local cache
- **Updates:** Rebuilt hourly from query logs

© Study Guide — For personal use only

11 Recommendation Engine

Two-tower, collaborative filtering

Two-Tower Architecture

- **Tower 1 (Candidate Generation):** Fast, coarse-grained; retrieves ~1000 candidate videos
- **Tower 2 (Ranking):** Slow, fine-grained; re-ranks top 100 using deep model
- **Advantage:** Separates scale (Tower 1) from quality (Tower 2)

Candidate Generation (Tower 1)

- **Method:** Collaborative filtering (user → user or item → item similarity)
- **Implementation:** User-video embedding space (Word2Vec-style on watch history)
- **Lookup:** Nearest-neighbor search in embedding space (using Approximate NN library)
- **Latency:** <50ms (cached results)

Ranking (Tower 2)

- **Model:** Deep neural network (user context + video features → CTR probability)
- **Features:** Watch history, user profile, video popularity, recency, seasonality
- **Output:** Click-through rate (CTR) prediction; rank by expected CTR
- **Latency:** <200ms for 100 videos; batch scoring

> ✓ **TIP**
>
> Recommendation latency target: <500ms (p99). Trade-off: faster = fewer candidates, lower quality.

© Study Guide — For personal use only

12 Comments & Engagement

Threading, reply-to, pagination

Comment Storage Architecture

- **Database:** Cassandra (distributed, write-optimized)
- **Sharding:** By video_id; allows efficient range queries (recent comments first)
- **Denormalization:** Store comment + author info in one row (avoid joins)
- **Replication:** 3 copies; quorum reads (2) + writes (2) for consistency

Comment Thread Model

```
Top-level comment structure:
comment_id: unique
video_id: sharding key
author_id: who wrote
text: comment body
created_at: timestamp
reply_count: cached counter
like_count: cached counter

Replies stored in separate table:
reply_id: unique
parent_comment_id: foreign key
author_id, text, created_at, like_count
```

Pagination & Sorting

- **Top comments:** Sorted by like_count (cached in Redis); refreshed every 5 min
- **Recent comments:** Sorted by created_at DESC (native Cassandra ordering)
- **Cursor pagination:** Use offset + limit for stability; cursor = last_comment_id
- **Latency:** <100ms for 20 comments per page

© Study Guide — For personal use only

13 Caching Strategy

Multi-layer cache hierarchy

Cache Layers

Layer	Technology	TTL	Hit Rate	Use Case
L1 Browser	LocalStorage / IndexedDB	1-7 days	90%+	Manifest, segments cache
L2 CDN Edge	Memcached / in-memory	1-7 days	95%+	Popular segment cache
L3 Regional	Memcached cluster	1-7 days	95%+	Fallback for edge miss
L4 Application	Redis (in-process)	1 hour	98%+	Metadata, session, counters
L5 Database	MySQL / Cassandra	N/A	N/A	Source of truth

Cache Invalidation Strategy

- **TTL-based:** Most common; safe but may serve stale data
- **Event-driven:** Publish to Kafka when metadata updated; subscribers invalidate
- **Hybrid:** Short TTL (5-10 min) + event-based invalidation for critical data

■ WARNING

Cache stampede: if popular item expires, many requests hit database simultaneously. Mitigation: **probabilistic early expiration** or **distributed locking**.

© Study Guide — For personal use only

14 Failure Modes & Resilience

Degradation, fallbacks, SLO targets

Key Failure Scenarios

- **Database shard down:** Replica takes over (RTO <10s); alert ops team
- **CDN PoP offline:** Geo-DNS reroutes traffic to next-nearest PoP; users see <200ms latency bump
- **Transcoding cluster overloaded:** Queue uploads; SLA slip to 8-12 hours (acceptable)
- **Streaming Service degradation:** Fall back to origin pull (slower but available); CDN load increases
- **Search outage:** Disable search UI; show trending videos instead
- **Recommendation model stale:** Use fallback heuristic (popular-by-category)

Circuit Breaker Pattern

```
if (search_service.failures > 10 in 30s) {
circuit_breaker.trip()
return fallback_heuristic() # popular videos
alert("Search service unhealthy")
}
// Retry with exponential backoff
for (attempt = 1 to 5) {
wait(100ms * 2^attempt)
if (search_service.health_check()) {
circuit_breaker.reset()
break
}
}
```

SLO Targets

Service	Latency (p99)	Availability
Streaming API	<100ms	99.99%
Upload Service	<500ms	99.9%
Search	<150ms	99.9%
Recommendations	<500ms	99.95%
CDN Segment Fetch	<50ms	99.99%

© Study Guide — For personal use only

15 Trade-Offs & Design Decisions

Strong vs Eventual Consistency

- **Choice:** MySQL (strong) for videos/users; Cassandra (eventual) for comments
- **Rationale:** Video metadata must be consistent (avoid serving mismatched manifests). Comments can be eventually consistent.
- **Trade-off:** MySQL slower to write but guarantees correctness; Cassandra fast for writes, OK with stale reads

Synchronous vs Asynchronous Encoding

- **Choice:** Asynchronous (queue-based) transcoding
- **Rationale:** Encoding 1TB takes 4 hours; blocking user on sync encode is unacceptable
- **Trade-off:** Users see 'processing' state; can watch in lower quality once first encode finishes

Centralized vs Distributed Transcoding

- **Choice:** Distributed (nodes in multiple regions)
- **Rationale:** Reduce latency for CDN warm-up; isolate regional failures; load balance across geographies
- **Trade-off:** Operational complexity (regional coordination, data transfer); but worth it at scale

Multi-Codec vs Single Codec

- **Choice:** Multi-codec (H.264 primary, VP9 secondary, HEVC for Apple)
- **Rationale:** Device compatibility; bandwidth savings (VP9 ~30% less, HEVC ~40% less)
- **Trade-off:** 3× storage & encoding cost; but justified by bandwidth savings for popular content

© Study Guide — For personal use only

16 Interview Playbook

How to ace the discussion

Opening (First 5 Minutes)

1. Clarify scope: 'Are we designing upload, playback, or both? What about live?'
2. Estimate scale: 'Let's assume 2B users, 500 hours/min upload, 1B hours/day watched.'
3. State assumptions: 'I'll focus on on-demand playback; we can discuss live as a follow-up.'

Architecture Design (Next 15-20 Minutes)

1. Draw high-level diagram: client → CDN → API → services → databases
2. Separate read path (CDN + streaming) from write path (upload + transcode)
3. Explain core trade-off: **write-once, read-many** → optimize for playback latency
4. Name key components: API gateway, upload service, streaming service, transcoding, message queue
5. Mention database choices: MySQL for metadata (consistency), Cassandra for comments (scale)

Deep Dives (Next 15 Minutes, Interviewer's Choice)

- If they ask about upload: discuss TUS protocol, chunking, resumability, deduplication
- If they ask about streaming: explain HLS/DASH, ABR algorithms (BOLA), CDN cache hierarchy
- If they ask about transcoding: codec comparison, resolution ladder, cost optimization, parallel workers
- If they ask about search: Elasticsearch, ranking formula, autocomplete, prefix trie
- If they ask about recommendations: two-tower model, candidate generation, ranking model, CTR prediction

Resilience & Trade-Offs (Final 5-10 Minutes)

1. Discuss failure modes: CDN down, database shard down, transcoding overload
2. Explain fallback strategies: circuit breakers, degraded modes, SLO-based prioritization
3. Compare trade-offs: strong vs eventual consistency, sync vs async, multi-codec vs single

Common Pitfalls

- ■ **Not clarifying scope:** Treat live streaming and on-demand the same (very different!)
- ■ **Overcomplicating early:** Jump to database sharding before discussing API design
- ■ **Ignoring CDN:** Design assumes all content from origin; miss ~90% of optimization
- ■ **Single codec:** Ignore bandwidth savings from VP9 / HEVC

© Study Guide — For personal use only

- ■ **No monitoring:** No mention of SLOs, metrics, or alerting

> ✓ **TIP**
>
> End strong: mention one recent innovation (e.g., 'I'd explore AV1 for premium content if encoding cost improves') or a metric you'd track (e.g., 'I'd monitor cache hit ratio by region to optimize CDN warm-up').

© Study Guide — For personal use only